Associate Web Instructor - Hiring Task

This assessment evaluates your engineering ability, judgment, technical communication, and teaching perception. You will build a full-stack feature, followed by producing two short video lessons (more on that later).
Before starting anything read the whole document from top to bottom.

Part 1: The project

Build a simple project submission and review dashboard. Initially you can keep your GitHub repo private. But after the submission deadline you must make your repository public. If your repository is not reachable then your submission will be disqualified.

Technical Stack:

- Programming Language: TypeScript
- Frontend: Next.js, TailwindCSS, ShadCN
- Backend: Express.js
- Database: PostgreSQL via Prisma ORM
- Auth: JWT based Role-Based Access Control

Feature Requirement:

1. Submission Portal (Student View)
    a. A form to submit project details (Project name, Repo URL, Live Link, Description)
    b. Implement robust validation and prevent duplicate submission
    c. Implement a basic rate limit to prevent spam
2. Review Dashboard (Mentor View)
    a. Paginated Data Table
    b. Approval and rejection action
    c. Filter to show "Approved" and "Rejected" projects (Filters should be implemented in server side)
    d. Aggregated statistics (e.g. Total Pending, Total Approved)
3. Authentication Page
    a. Login page
    b. Signup page for student
    c. For now you can seed the admin directly in the database

Note: There are some ambiguities left in the feature requirement intentionally. Use your best judgment to fill in.

Part 2: Video Contents

Below there are two categories given. You need to act as a mentor and create 2 videos in Bengali for your students teaching them the topics. Each video should be between 10 to 12 minutes max. You need to use OBS in order to record the videos. Your face must be visible in the video. Setup the OBS in a way that your facecam is in the lower right corner of the screen. (Don't worry about the facecam resolution. For now you can use a normal webcam or your phone.)

There are two categories: "Theoretical Video" and "Hands-on coding demonstration video". You need to submit 2 videos, one from each category (e.g 1C, 2B). Be very careful about this, if you choose two topics from the same category (e.g. 1B, 1D) then your submission will be disqualified immediately.

The topics are related to the project itself. So you don't need to prepare a separate demo project for the videos. Instead you need to use the project as teaching material. For example if you are going to explain the next.js render strategies, use your project as teaching material and explain what you did and why you did it that way.

Category 1 - Theoretical Video

Explain a web development concept simply and clearly, focusing on the "why" rather than just the syntax. You are not obligated to show code implementation. You can use diagrams, images, presentation slides, and tablet if you want to. (Choose only one topic from below)

Topic 1A = The N+1 Query Problem: Explain what the N+1 problem is in ORMs (like Prisma or Drizzle), why it degrades database performance as an application scales, and the theoretical approach to solving it.

Topic 1B = The Architecture of Caching: Explain why a high-traffic web application might need a caching layer (like Redis) in front of a robust database like PostgreSQL, using the dashboard statistics feature as a real-world analogy.

Topic 1C = Authentication vs. Authorization (RBAC)**:** Demystify the conceptual difference between verifying identity (AuthN) and Role-Based Access Control (AuthZ). Explain how a server securely differentiates between a Student and a Mentor without compromising performance.

Topic 1D = Next.js Rendering Strategies: Break down the difference between Server Components and Client Components in the Next.js App Router. Explain the performance and security trade-offs, and why a developer would choose one over the other for a data submission form.

Category 2 - Hands on coding demonstration video

Demonstrate your ability to write clean code live, explain your reasoning, and anticipate beginner mistakes. (Choose only one topic from below)

Topic 2A = Securing a Route: Live code a backend route handler. Demonstrate how to extract the user ID from an auth token, query the database, and definitively verify that the user has the right access to resource before allowing a status update.

Topic 2B = Database Schema & Relations: Live code the database schema creation using Prisma. Explain the one-to-many or many-to-many relations between Users, Submissions, and Feedback, and explain your thought process while modeling the data.

Topic 2C = State Management & Data Fetching: Live code the implementation of data fetching for the Mentor Dashboard. Explain how to elegantly handle loading states, cache invalidation, and UI error boundaries.

Topic 2D = Defensive Form Validation: Live code a robust form submission flow. Show how to implement strict validation (e.g., using Zod) to prevent malformed data or duplicate submissions, and demonstrate how to surface those validation errors gracefully to the user in the frontend.

Submission Guideline

We will share a google form with you and you will submit your task there. You need to submit

1. **GitHub Repository Link**
2. **Two video links (Unlisted YouTube video or Google Drive with open view permission)**

All of the tasks mentioned in this document are required. There are no optional tasks. Missing any one of them will result in point deduction.

Thank you and best of luck.